

JAVA ERRORS

Finding & Fixing Code Faults

codehive

codehive

Types of Errors

In Java, errors are classified into three main categories based on when they occur and what causes them.

Error Type	When it Occurs	Difficulty
Compile-Time	During code compilation	Easy (Fixed by IDE)
Runtime	During program execution	Medium (Crashes)
Logical	During execution (Wrong result)	Hard (Silent)

1. Compile-Time Errors

These are detected by the Java compiler before your program even starts. If these exist, the code won't run.

Missing Semicolons

```
int x = 5  
System.out.println(x);
```

error: ';' expected

Undeclared Variables

```
System.out.println(myValue);
```

error: cannot find symbol: variable myValue

Pro Tip: Modern IDEs like IntelliJ or Eclipse will underline these in red while you type!

2. Runtime Errors

Your program compiles and starts, but a specific situation causes it to "crash" or throw an exception.

Type Mismatches

```
int x = "Hello";
```

incompatible types: String cannot be converted to int

Arithmetic Errors

The most famous runtime error is **Division by Zero**.

```
int result = 10 / 0;
```

Exception in thread "main" java.lang.ArithmeticException: / by zero

3. Array & Index Errors

Accessing data that doesn't exist is a common way to trigger a runtime crash.

Index Out of Bounds

```
int[] numbers = {1, 2, 3};  
System.out.println(numbers[8]); // Index 8 doesn't exist
```

```
java.lang.ArrayIndexOutOfBoundsException: Index 8 out of bounds
```

Deep Dive: Java arrays are 0-indexed. This means a length-3 array only has indices 0, 1, and 2.

4. Logical Errors

Logical errors are the most dangerous because the program runs without any error messages, but the result is **wrong**.

```
int x = 10;
int y = 2;
int sum = x - y; // Intended addition, wrote subtraction
System.out.println("x + y = " + sum);
```

Attempted Result	12
Actual Output	8

Logical errors require careful testing and debugging with different input values.

Developer Best Practices

To minimize time spent debugging, follow these habits used by professional developers:

- ✓ **Read the Stack Trace:** Java error messages tell you the exactly file and line number where things went wrong.
- ✓ **Use Meaningful Names:** Don't name variables `a`, `b`, `c`. Use `studentAge`.
- ✓ **Consistent Indentation:** Keeping your code formatted makes missing braces `{ }` obvious.
- ✓ **Small Samples:** Test individual parts of your code frequently rather than writing 500 lines at once.

End of Debugging Manual

codehive

Turning Errors into Lessons.